

AET_NN_Colab

June 11, 2025

```
[1]: import xarray as xr
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from datetime import datetime
import pandas as pd
from glob import glob
```

LSTM: - sequence length = 30 - features = PCA - loss = MSE - optimizer = Adam - batch size = 64

```
[2]: # load the processed data from the CSV file
loaded_data = pd.read_csv('PCA_processed_climatological_data.csv')
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
<ipython-input-2-2364396700> in <cell line: 0>()
      1 # load the processed data from the CSV file
----> 2 loaded_data = pd.read_csv('PCA_processed_climatological_data.csv')

/usr/local/lib/python3.11/dist-packages/pandas/io/parsers/readers.py in
    ↪read_csv(filepath_or_buffer, sep, delimiter, header, names, index_col,
    ↪usecols, dtype, engine, converters, true_values, false_values,
    ↪skipinitialspace, skiprows, skipfooter, nrows, na_values, keep_default_na,
    ↪na_filter, verbose, skip_blank_lines, parse_dates, infer_datetime_format,
    ↪keep_date_col, date_parser, date_format, dayfirst, cache_dates, iterator,
    ↪chunksize, compression, thousands, decimal, lineterminator, quotechar,
    ↪quoting, doublequote, escapechar, comment, encoding, encoding_errors, dialect,
    ↪on_bad_lines, delim_whitespace, low_memory, memory_map, float_precision,
    ↪storage_options, dtype_backend)
    1024         kwds.update(kwds_defaults)
    1025
-> 1026     return _read(filepath_or_buffer, kwds)
    1027
    1028

/usr/local/lib/python3.11/dist-packages/pandas/io/parsers/readers.py in
    ↪_read(filepath_or_buffer, kwds)
    618
    619     # Create the parser.
--> 620     parser = TextFileReader(filepath_or_buffer, **kwds)
```

```

621
622     if chunksize or iterator:

/usr/local/lib/python3.11/dist-packages/pandas/io/parsers/readers.py in
↳ __init__(self, f, engine, **kwds)
1618
1619         self.handles: IOHandles | None = None
-> 1620         self._engine = self._make_engine(f, self.engine)
1621
1622     def close(self) -> None:

/usr/local/lib/python3.11/dist-packages/pandas/io/parsers/readers.py in
↳ _make_engine(self, f, engine)
1878         if "b" not in mode:
1879             mode += "b"
-> 1880         self.handles = get_handle(

1881             f,
1882             mode,

/usr/local/lib/python3.11/dist-packages/pandas/io/common.py in
↳ get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text,
↳ errors, storage_options)
871         if ioargs.encoding and "b" not in ioargs.mode:
872             # Encoding
--> 873             handle = open(

874                 handle,
875                 ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory:
↳ 'PCA_processed_climatological_data.csv'

```

```

[ ]: *** build the sequences **

# select unique locations based on longitude and latitude
unique_locations = loaded_data[['longitude', 'latitude']].drop_duplicates()

X_sequences = []
y_sequences = []

sequence_length = 30

for _, loc in unique_locations.iterrows():
    loc_data = loaded_data[
        (loaded_data['longitude'] == loc['longitude']) &
        (loaded_data['latitude'] == loc['latitude'])
    ].sort_values(by='elapsed_days')

```

```

features = loc_data[['PC1', 'PC2', 'PC3', 'PC4', 'PC5', 'PC6']].values
target = loc_data['E'].values

for i in range(len(loc_data) - sequence_length):
    X_seq = features[i:i+sequence_length]
    y_val = target[i+sequence_length]
    X_sequences.append(X_seq)
    y_sequences.append(y_val)

X = np.array(X_sequences)
y = np.array(y_sequences)

```

```
[ ]: *** build a dataset of shape (n_samples, n_features) **
```

```

print(f"Feature matrix shape: {X.shape}")
print(f"Target vector shape: {y.shape}")
print(X[0][:5])
print(y[0])

```

```

Feature matrix shape: (156252, 30, 6)
Target vector shape: (156252,)
[[-1.08733263 -1.30367916  0.81712897 -0.70372497 -0.51489596 -0.79129969]
 [-0.46048293 -2.71307528  1.40493803  0.05134136  0.41994429 -1.01419211]
 [-0.83721439 -1.74891164  1.62980053 -0.1474909  -0.19433768 -1.1331816 ]
 [-0.90566018 -0.83460412  0.83771119 -0.72184927 -0.35064262 -1.29424872]
 [-1.16611563 -0.20218523  0.3233643  -1.05541314 -0.98714769 -1.16150876]]
-0.85567915

```

```
[ ]: *** mix the sequences of data **
```

```

from sklearn.utils import shuffle

X, y = shuffle(X, y, random_state=42)

print(f"Feature matrix shape after shuffle: {X.shape}")
print(f"Target vector shape after shuffle: {y.shape}")
print(X[0][:5])
print(y[0])

```

```

Feature matrix shape after shuffle: (156252, 30, 6)
Target vector shape after shuffle: (156252,)
[[ 2.04267063  1.83741034  1.28269879  0.15353406 -1.43182713 -0.20545752]
 [ 2.38098807  1.32357226  0.45579041 -0.01397663 -1.02923179 -0.05667657]
 [ 2.76650384  0.79649688  0.10760745  0.10039634 -0.20102059 -0.4325268 ]
 [ 3.17929062  0.56892787  0.63210398  0.28048847 -0.44728061 -0.08590704]
 [ 3.06376799  0.40301045 -0.15557128  0.10679341  0.15697895 -0.42027523]]
-1.1966311

```

```
[ ]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42)

print(f"Training set shape: {X_train.shape}, {y_train.shape}")
print(f"Testing set shape: {X_test.shape}, {y_test.shape}")
```

Training set shape: (125001, 30, 6), (125001,)
Testing set shape: (31251, 30, 6), (31251,)

```
[ ]: import tensorflow as tf

print(tf.config.list_physical_devices('GPU'))

# the model learns to predict the next day ET value based on the previous 30
↳days of data

model = tf.keras.Sequential()

model.add(tf.keras.layers.Input(shape=(sequence_length, 6))) # input shape
↳(sequence_length, n_features)
model.add(tf.keras.layers.LSTM(128, return_sequences=True)) #
↳return_sequences=True, each of the LSTM cells returns its sequence of 128
↳outputs. So the output shape is (batch_size, sequence_length, 128)

model.add(tf.keras.layers.LSTM(64, return_sequences=False)) #
↳return_sequences=False, the last LSTM cell returns its last output. So the
↳output shape is (batch_size, 64)
model.add(tf.keras.layers.Dropout(0.1))

model.add(tf.keras.layers.Dense(64, activation='relu'))

model.add(tf.keras.layers.Dense(1)) # predict a single ET value

# compile the model
model.compile(optimizer='adam', loss='mse')
model.summary()
```

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 30, 128)	69,120

lstm_1 (LSTM)	(None, 64)	49,408
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 64)	4,160
dense_1 (Dense)	(None, 1)	65

Total params: 122,753 (479.50 KB)

Trainable params: 122,753 (479.50 KB)

Non-trainable params: 0 (0.00 B)

```
[ ]: import time

start_time = time.time()

history = model.fit(
    X_train, y_train,
    batch_size=64,
    validation_split=0.2,
    epochs=50,
)

end_time = time.time()
print(f"Training time: {end_time - start_time:.2f} seconds")
```

```
Epoch 1/75
1563/1563          17s 8ms/step -
loss: 0.2343 - val_loss: 0.2006
Epoch 2/75
1563/1563          18s 8ms/step -
loss: 0.1814 - val_loss: 0.1772
Epoch 3/75
1563/1563          20s 8ms/step -
loss: 0.1678 - val_loss: 0.1629
Epoch 4/75
1563/1563          20s 8ms/step -
loss: 0.1518 - val_loss: 0.1546
Epoch 5/75
1563/1563          22s 9ms/step -
loss: 0.1395 - val_loss: 0.1444
Epoch 6/75
1563/1563          19s 8ms/step -
```

loss: 0.1293 - val_loss: 0.1354
Epoch 7/75
1563/1563 20s 8ms/step -
loss: 0.1192 - val_loss: 0.1294
Epoch 8/75
1563/1563 23s 9ms/step -
loss: 0.1105 - val_loss: 0.1177
Epoch 9/75
1563/1563 12s 8ms/step -
loss: 0.1029 - val_loss: 0.1061
Epoch 10/75
1563/1563 20s 8ms/step -
loss: 0.0958 - val_loss: 0.1051
Epoch 11/75
1563/1563 12s 8ms/step -
loss: 0.0892 - val_loss: 0.1027
Epoch 12/75
1563/1563 22s 9ms/step -
loss: 0.0827 - val_loss: 0.0928
Epoch 13/75
1563/1563 13s 8ms/step -
loss: 0.0779 - val_loss: 0.0932
Epoch 14/75
1563/1563 12s 8ms/step -
loss: 0.0712 - val_loss: 0.0917
Epoch 15/75
1563/1563 12s 8ms/step -
loss: 0.0673 - val_loss: 0.0825
Epoch 16/75
1563/1563 12s 8ms/step -
loss: 0.0635 - val_loss: 0.0912
Epoch 17/75
1563/1563 21s 8ms/step -
loss: 0.0612 - val_loss: 0.0780
Epoch 18/75
1563/1563 20s 8ms/step -
loss: 0.0552 - val_loss: 0.0766
Epoch 19/75
1563/1563 12s 8ms/step -
loss: 0.0546 - val_loss: 0.0748
Epoch 20/75
1563/1563 20s 8ms/step -
loss: 0.0516 - val_loss: 0.0741
Epoch 21/75
1563/1563 21s 8ms/step -
loss: 0.0482 - val_loss: 0.0784
Epoch 22/75
1563/1563 20s 8ms/step -

loss: 0.0485 - val_loss: 0.0749
Epoch 23/75
1563/1563 20s 8ms/step -
loss: 0.0437 - val_loss: 0.0737
Epoch 24/75
1563/1563 12s 8ms/step -
loss: 0.0435 - val_loss: 0.0712
Epoch 25/75
1563/1563 21s 8ms/step -
loss: 0.0414 - val_loss: 0.0661
Epoch 26/75
1563/1563 21s 8ms/step -
loss: 0.0372 - val_loss: 0.0650
Epoch 27/75
1563/1563 20s 8ms/step -
loss: 0.0361 - val_loss: 0.0672
Epoch 28/75
1563/1563 21s 8ms/step -
loss: 0.0375 - val_loss: 0.0649
Epoch 29/75
1563/1563 12s 8ms/step -
loss: 0.0328 - val_loss: 0.0660
Epoch 30/75
1563/1563 21s 8ms/step -
loss: 0.0328 - val_loss: 0.0728
Epoch 31/75
1563/1563 20s 8ms/step -
loss: 0.0368 - val_loss: 0.0639
Epoch 32/75
1563/1563 21s 8ms/step -
loss: 0.0295 - val_loss: 0.0710
Epoch 33/75
1563/1563 12s 8ms/step -
loss: 0.0334 - val_loss: 0.0591
Epoch 34/75
1563/1563 20s 8ms/step -
loss: 0.0274 - val_loss: 0.0633
Epoch 35/75
1563/1563 21s 8ms/step -
loss: 0.0309 - val_loss: 0.0683
Epoch 36/75
1563/1563 20s 8ms/step -
loss: 0.0273 - val_loss: 0.0613
Epoch 37/75
1563/1563 20s 8ms/step -
loss: 0.0255 - val_loss: 0.0617
Epoch 38/75
1563/1563 21s 8ms/step -

```

loss: 0.0252 - val_loss: 0.0616
Epoch 39/75
1563/1563          20s 8ms/step -
loss: 0.0304 - val_loss: 0.0614
Epoch 40/75
1563/1563          12s 8ms/step -
loss: 0.0238 - val_loss: 0.0604
Epoch 41/75
1563/1563          12s 8ms/step -
loss: 0.0236 - val_loss: 0.0634
Epoch 42/75
1563/1563          14s 9ms/step -
loss: 0.0235 - val_loss: 0.0629
Epoch 43/75
1563/1563          19s 8ms/step -
loss: 0.0233 - val_loss: 0.0646
Epoch 44/75
1563/1563          20s 8ms/step -
loss: 0.0233 - val_loss: 0.0613
Epoch 45/75
1563/1563          21s 8ms/step -
loss: 0.0201 - val_loss: 0.0581
Epoch 46/75
1563/1563          20s 8ms/step -
loss: 0.0204 - val_loss: 0.0597
Epoch 47/75
1563/1563          12s 8ms/step -
loss: 0.0201 - val_loss: 0.0611
Epoch 48/75
1563/1563          13s 9ms/step -
loss: 0.0226 - val_loss: 0.0587
Epoch 49/75
1563/1563          12s 8ms/step -
loss: 0.0192 - val_loss: 0.0577
Epoch 50/75
1563/1563          20s 8ms/step -
loss: 0.0187 - val_loss: 0.0613
Epoch 51/75
1563/1563          20s 8ms/step -
loss: 0.0254 - val_loss: 0.0590
Epoch 52/75
1563/1563          12s 8ms/step -
loss: 0.0193 - val_loss: 0.0575
Epoch 53/75
1563/1563          12s 8ms/step -
loss: 0.0184 - val_loss: 0.0598
Epoch 54/75
1563/1563          12s 8ms/step -

```



```

loss: 0.0199 - val_loss: 0.0589
Epoch 55/75
1563/1563          21s 8ms/step -
loss: 0.0174 - val_loss: 0.0595
Epoch 56/75
1563/1563          20s 8ms/step -
loss: 0.0170 - val_loss: 0.0575
Epoch 57/75
1563/1563          20s 8ms/step -
loss: 0.0174 - val_loss: 0.0587
Epoch 58/75
1563/1563          20s 8ms/step -
loss: 0.0167 - val_loss: 0.0632
Epoch 59/75
1563/1563          20s 8ms/step -
loss: 0.0195 - val_loss: 0.0569
Epoch 60/75
1563/1563          22s 9ms/step -
loss: 0.0167 - val_loss: 0.0649
Epoch 61/75
1563/1563          12s 8ms/step -
loss: 0.0176 - val_loss: 0.0579
Epoch 62/75
1563/1563          12s 8ms/step -
loss: 0.0149 - val_loss: 0.0591
Epoch 63/75
1563/1563          23s 9ms/step -
loss: 0.0150 - val_loss: 0.0584
Epoch 64/75
1563/1563          13s 8ms/step -
loss: 0.0149 - val_loss: 0.0598
Epoch 65/75
1563/1563          20s 8ms/step -
loss: 0.0151 - val_loss: 0.0609
Epoch 66/75
1563/1563          21s 8ms/step -
loss: 0.0154 - val_loss: 0.0571
Epoch 67/75
1563/1563          20s 8ms/step -
loss: 0.0144 - val_loss: 0.0618
Epoch 68/75
1563/1563          21s 9ms/step -
loss: 0.0144 - val_loss: 0.0595
Epoch 69/75
1563/1563          19s 8ms/step -
loss: 0.0139 - val_loss: 0.0577
Epoch 70/75
1563/1563          20s 8ms/step -

```

```

loss: 0.0139 - val_loss: 0.0600
Epoch 71/75
1563/1563          20s 8ms/step -
loss: 0.0136 - val_loss: 0.0658
Epoch 72/75
1563/1563          12s 8ms/step -
loss: 0.0178 - val_loss: 0.0586
Epoch 73/75
1563/1563          12s 8ms/step -
loss: 0.0126 - val_loss: 0.0575
Epoch 74/75
1563/1563          12s 8ms/step -
loss: 0.0138 - val_loss: 0.0580
Epoch 75/75
1563/1563          12s 8ms/step -
loss: 0.0135 - val_loss: 0.0590
Training time: 1334.57 seconds

```

```

[ ]: # save the model
model.save('PCA_et_prediction_model.h5')

```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

LSTM: - sequence length = 30 - random missing input features - loss = MSE - optimizer = Adam
- batch size = 64

```

[32]: # load the processed data from the CSV file
reduced_dataset = pd.read_csv('processed_climatological_data_with_nan.csv')

```

```

[34]: *** build the sequences **

unique_locations = reduced_dataset[['longitude', 'latitude']].drop_duplicates()

X_sequences = []
y_sequences = []

sequence_length = 30

feature_cols = ['PC1', 'PC2', 'PC3', 'PC4', 'PC5', 'PC6', 'PC7']

for _, loc in unique_locations.iterrows():
    loc_data = reduced_dataset[
        (reduced_dataset['longitude'] == loc['longitude']) &
        (reduced_dataset['latitude'] == loc['latitude'])

```

```

].sort_values(by='elapsed_days')

features = loc_data[feature_cols].values
target = loc_data['E'].values

for i in range(len(loc_data) - sequence_length):
    X_seq = features[i:i+sequence_length]
    y_val = target[i+sequence_length]
    X_sequences.append(X_seq)
    y_sequences.append(y_val)

X = np.array(X_sequences)
y = np.array(y_sequences)

```

[35]: **** build a dataset of shape (n_samples, n_features) ***

```

print(f"Feature matrix shape: {X.shape}")
print(f"Target vector shape: {y.shape}")
print(X[0][:5])
print(y[0])

```

Feature matrix shape: (156252, 30, 7)

Target vector shape: (156252,)

```

[[-0.82978674 -1.1431502  0.85141805 -0.71261053 -0.52306202 -0.92820815
  -0.80250841]
 [-0.46291794 -2.71383335  1.39816482  0.04726615  0.41452445 -1.01420357
  -0.85581847]
 [-0.83856147 -1.75102817  1.62255607 -0.15012411 -0.2020182  -1.13375323
  -0.2795174 ]
 [-0.90585532 -0.83496617  0.83117273 -0.72312738 -0.35686555 -1.29353678
  -0.11906471]
 [-1.04451891 -0.59217946  0.47970007 -0.83641223 -0.57171213 -1.38216061
  -0.19799387]]
-0.85567915

```

[36]: **** mix the sequences of data ***

```

from sklearn.utils import shuffle

X, y = shuffle(X, y, random_state=42)

print(f"Feature matrix shape after shuffle: {X.shape}")
print(f"Target vector shape after shuffle: {y.shape}")
print(X[0][:5])
print(y[0])

```

Feature matrix shape after shuffle: (156252, 30, 7)

Target vector shape after shuffle: (156252,)

```

[[ 2.43321138  1.04696042  1.25065575  0.32883979 -1.12567931  0.16865034

```

```

-0.03325949]
[ 1.81235432  1.15355816  0.48066518  0.0146109 -1.060755  0.07227551
 0.79456471]
[ 2.76895747  0.79391619  0.1091791  0.10034533 -0.20226925 -0.43124885
 0.90621694]
[ 3.18126173  0.56411106  0.63188215  0.28027287 -0.45049244 -0.08414317
 0.59176173]
[ 3.06594505  0.40139236 -0.15272872  0.10634924  0.15792748 -0.41950574
 1.07964128]]
-1.1966311

```

```

[37]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42)

print(f"Training set shape: {X_train.shape}, {y_train.shape}")
print(f"Testing set shape: {X_test.shape}, {y_test.shape}")

```

```

Training set shape: (125001, 30, 7), (125001,)
Testing set shape: (31251, 30, 7), (31251,)

```

```

[38]: import tensorflow as tf

print(tf.config.list_physical_devices('GPU'))

# the model learns to predict the next day ET value based on the previous 30
↳days of data

model = tf.keras.Sequential()

model.add(tf.keras.layers.Input(shape=(sequence_length, 7))) # input shape
↳(sequence_length, n_features)
model.add(tf.keras.layers.LSTM(128, return_sequences=True)) #
↳return_sequences=True, each of the LSTM cells returns its sequence of 128
↳outputs. So the output shape is (batch_size, sequence_length, 128)

model.add(tf.keras.layers.LSTM(64, return_sequences=False)) #
↳return_sequences=False, the last LSTM cell returns its last output. So the
↳output shape is (batch_size, 64)
model.add(tf.keras.layers.Dropout(0.1))

model.add(tf.keras.layers.Dense(64, activation='relu'))

model.add(tf.keras.layers.Dense(1)) # predict a single ET value

model.compile(optimizer='adam', loss='mse')
model.summary()

```

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

```
Model: "sequential_5"
```

Layer (type)	Output Shape	Param #
lstm_9 (LSTM)	(None, 30, 128)	69,632
lstm_10 (LSTM)	(None, 64)	49,408
dropout_4 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 64)	4,160
dense_9 (Dense)	(None, 1)	65

```
Total params: 123,265 (481.50 KB)
```

```
Trainable params: 123,265 (481.50 KB)
```

```
Non-trainable params: 0 (0.00 B)
```

```
[39]: import time

start_time = time.time()

history = model.fit(
    X_train, y_train,
    batch_size=64,
    validation_split=0.2,
    epochs=50,
)

end_time = time.time()
print(f"Training time: {end_time - start_time:.2f} seconds")
```

```
Epoch 1/50
1563/1563          15s 8ms/step -
loss: 0.2350 - val_loss: 0.1900
Epoch 2/50
1563/1563          12s 8ms/step -
loss: 0.1798 - val_loss: 0.1598
Epoch 3/50
1563/1563          12s 8ms/step -
```

```

loss: 0.1567 - val_loss: 0.1571
Epoch 4/50
1563/1563          22s 9ms/step -
loss: 0.1450 - val_loss: 0.1433
Epoch 5/50
1563/1563          12s 8ms/step -
loss: 0.1322 - val_loss: 0.1396
Epoch 6/50
1563/1563          21s 8ms/step -
loss: 0.1218 - val_loss: 0.1236
Epoch 7/50
1563/1563          20s 8ms/step -
loss: 0.1125 - val_loss: 0.1169
Epoch 8/50
1563/1563          12s 8ms/step -
loss: 0.1057 - val_loss: 0.1204
Epoch 9/50
1563/1563          20s 8ms/step -
loss: 0.0990 - val_loss: 0.1015
Epoch 10/50
1563/1563          20s 8ms/step -
loss: 0.0891 - val_loss: 0.1050
Epoch 11/50
1563/1563          21s 8ms/step -
loss: 0.0882 - val_loss: 0.0968
Epoch 12/50
1563/1563          20s 8ms/step -
loss: 0.0791 - val_loss: 0.0925
Epoch 13/50
1563/1563          20s 8ms/step -
loss: 0.0751 - val_loss: 0.0885
Epoch 14/50
1563/1563          20s 8ms/step -
loss: 0.0705 - val_loss: 0.0946
Epoch 15/50
1563/1563          21s 8ms/step -
loss: 0.0714 - val_loss: 0.0888
Epoch 16/50
1563/1563          21s 8ms/step -
loss: 0.0625 - val_loss: 0.0832
Epoch 17/50
1563/1563          12s 8ms/step -
loss: 0.0589 - val_loss: 0.0809
Epoch 18/50
1563/1563          21s 8ms/step -
loss: 0.0562 - val_loss: 0.0847
Epoch 19/50
1563/1563          21s 8ms/step -

```

```

loss: 0.0561 - val_loss: 0.0816
Epoch 20/50
1563/1563          20s 8ms/step -
loss: 0.0503 - val_loss: 0.0824
Epoch 21/50
1563/1563          20s 8ms/step -
loss: 0.0513 - val_loss: 0.0792
Epoch 22/50
1563/1563          21s 8ms/step -
loss: 0.0461 - val_loss: 0.0777
Epoch 23/50
1563/1563          21s 8ms/step -
loss: 0.0428 - val_loss: 0.0839
Epoch 24/50
1563/1563          20s 8ms/step -
loss: 0.0430 - val_loss: 0.0805
Epoch 25/50
1563/1563          13s 8ms/step -
loss: 0.0407 - val_loss: 0.0779
Epoch 26/50
1563/1563          21s 8ms/step -
loss: 0.0383 - val_loss: 0.0765
Epoch 27/50
1563/1563          21s 9ms/step -
loss: 0.0367 - val_loss: 0.0775
Epoch 28/50
1563/1563          20s 8ms/step -
loss: 0.0359 - val_loss: 0.0765
Epoch 29/50
1563/1563          20s 8ms/step -
loss: 0.0341 - val_loss: 0.0758
Epoch 30/50
1563/1563          21s 8ms/step -
loss: 0.0332 - val_loss: 0.0774
Epoch 31/50
1563/1563          13s 8ms/step -
loss: 0.0332 - val_loss: 0.0762
Epoch 32/50
1563/1563          21s 9ms/step -
loss: 0.0307 - val_loss: 0.0759
Epoch 33/50
1563/1563          20s 8ms/step -
loss: 0.0298 - val_loss: 0.0738
Epoch 34/50
1563/1563          13s 8ms/step -
loss: 0.0291 - val_loss: 0.0759
Epoch 35/50
1563/1563          21s 8ms/step -

```

```

loss: 0.0277 - val_loss: 0.0785
Epoch 36/50
1563/1563          20s 8ms/step -
loss: 0.0307 - val_loss: 0.0730
Epoch 37/50
1563/1563          13s 8ms/step -
loss: 0.0259 - val_loss: 0.0770
Epoch 38/50
1563/1563          21s 9ms/step -
loss: 0.0256 - val_loss: 0.0756
Epoch 39/50
1563/1563          19s 8ms/step -
loss: 0.0253 - val_loss: 0.0725
Epoch 40/50
1563/1563          13s 8ms/step -
loss: 0.0255 - val_loss: 0.0739
Epoch 41/50
1563/1563          12s 8ms/step -
loss: 0.0232 - val_loss: 0.0774
Epoch 42/50
1563/1563          12s 8ms/step -
loss: 0.0233 - val_loss: 0.0759
Epoch 43/50
1563/1563          13s 8ms/step -
loss: 0.0225 - val_loss: 0.0746
Epoch 44/50
1563/1563          21s 8ms/step -
loss: 0.0221 - val_loss: 0.0765
Epoch 45/50
1563/1563          20s 8ms/step -
loss: 0.0213 - val_loss: 0.0738
Epoch 46/50
1563/1563          13s 8ms/step -
loss: 0.0213 - val_loss: 0.0751
Epoch 47/50
1563/1563          22s 9ms/step -
loss: 0.0207 - val_loss: 0.0773
Epoch 48/50
1563/1563          19s 8ms/step -
loss: 0.0204 - val_loss: 0.0763
Epoch 49/50
1563/1563          13s 8ms/step -
loss: 0.0197 - val_loss: 0.0783
Epoch 50/50
1563/1563          20s 8ms/step -
loss: 0.0198 - val_loss: 0.0746
Training time: 901.91 seconds

```



```
[41]: # save the model
      model.save('et_prediction_model_nanValues.h5')
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.